PRIORIS: CONTEXT AWARE TO DO APP USING PYTHON

MINOR PROJECT REPORT

By

ANUBHAV SHARMA (RA2411026030010)
TANISHK BHATNAGAR (RA2411026030018)
ASHISH S NAIR (RA2411026030022)

Under the guidance of

MS. SHRUTHY GOVINDAN

In partial fulfilment for the Course

of

21CSC203P – ADVANCED PROGRAMMING PRACTICE

in COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DELHI-NCR CAMPUS

NOVEMBER-2025

BONAFIDE CERTIFICATE

Certified that this minor project report for the course **21CSC203P ADVANCED PROGRAMMING PRACTICE** entitled in " **Prioris: Context-Aware To-do application using Python** " is the Bonafide work of **Anubhav Sharma (RA2411026030010), Tanishk Bhatnagar (RA2411026030018), Ashish S Nair (RA2411026030022)** who carried out the work under my supervision.

.

SIGNATURE

Ms. Shruthy Govindan
Assistant Professor
Computer Science and Engineering
SRM Institute of Science and Technology
Delhi-NCR Campus

ABSTRACT

PRIORIS is a context-aware to-do application built using Python, designed to intelligently manage daily tasks based on location, time, weather and focus context. Unlike traditional task managers, Prioris integrates Eisenhower Matrix prioritization, automates reminders and notifications using context triggers like geo-fencing, time windows, and device connectivity.

The project aims to create a smart, laptop-based productivity assistant that adjusts dynamically to the user's environment activating focus modes, commute reminders, and task suggestions in real-time. Through a modular and scalable architecture, Prioris allows for seamless feature integration and an adaptive, user-centric experience.

ACKNOWLEDGEMENT

We, **Ashish S Nair, Anubhav Sharma,** and **Tanishk Bhatnagar**, students of second year B.Tech Computer Science and Engineering AIML, express our deepest gratitude to our guide **Ms. Shruthy Govindan** for her continuous guidance and encouragement throughout this project.

We extend our appreciation to **Dr. Harish Kumar,** Head of Department, for his constant support and motivation. We also thank the faculty members, lab assistants, and our peers for their valuable insights during various stages of development.

Lastly, we thank our families and friends for their unwavering support and patience during the completion of this project.

TABLE OF CONTENTS

INTRODUCTION

The **PRIORIS** project is a Python-based intelligent task management system that adapts to a user's **context** — such as time, location, weather, and connectivity — to automate reminders and optimize productivity.
Developed as a desktop (laptop) application, Prioris introduces **context-awareness** to traditional to-do lists, reducing manual input and helping users stay consistent with daily goals.

1.1 Motivation

In an era of distractions and multitasking, productivity tools often fail to respond to the user's changing environment. Prioris bridges this gap by combining the principles of **Eisenhower matrix and smart suggestions** into one system that intelligently activates relevant tasks when conditions are met (e.g., entering "Home" triggers personal chores; "Work" activates professional tasks).

1.2 Objective

The primary objective of Prioris is to develop an adaptive task management system capable of:

- Develop a **Wi-Fi–based smart alert** system for context detection.
- Provide **deadline-based alerts** when due dates are near or exceeded.
- Integrate **weather-based recommendations** to assist in planning tasks.
- Maintain a **modular structure for easy expansion** and integration of future features.Providing a modular framework for easy expansion (more triggers or smart categories).

1.3 Problem Statement

Traditional to-do apps lack contextual intelligence and prioritize tasks purely by time. Prioris overcomes this limitation by introducing context awareness and Eisenhower prioritization, offering a smarter and more intuitive way to manage productivity.

1.4 Challenges

The development of Prioris involves several challenges:

- Integrating Wi-Fi detection reliably on a desktop environment.
- Combining multiple triggers (Wi-Fi, deadlines, weather) without performance lag
- Mapping tasks accurately within the Eisenhower framework dynamically.
- Ensuring data persistence and smooth UI performance.

REQUIREMENT ANALYSIS

2.1 Functional Requirements

1. **Task Management:**
    - Create, edit, and delete tasks with details such as deadlines, Wi-Fi zone, and priority category.

2. **Eisenhower Matrix Integration:**
    - Classify tasks into one of four quadrants:
        - **Urgent & Important**
        - **Important but Not Urgent**
        - **Urgent but Not Important**
        - **Neither Urgent nor Important**

3. **Wi-Fi Context Detection:**
    - Trigger relevant task sets when connected to predefined networks (e.g., “Home” or “Office”).

4. **Deadline-Based Alerts:**
    - Send reminders and alerts when a deadline is approaching or overdue.

5. **Weather-Based Alerts:**
    - Get location manually inputted from the user for particular tasks and fetch live weather data and generate context-aware suggestions (e.g., “It might rain for the next few days, get your task done now”).

2.2 Technical Requirements

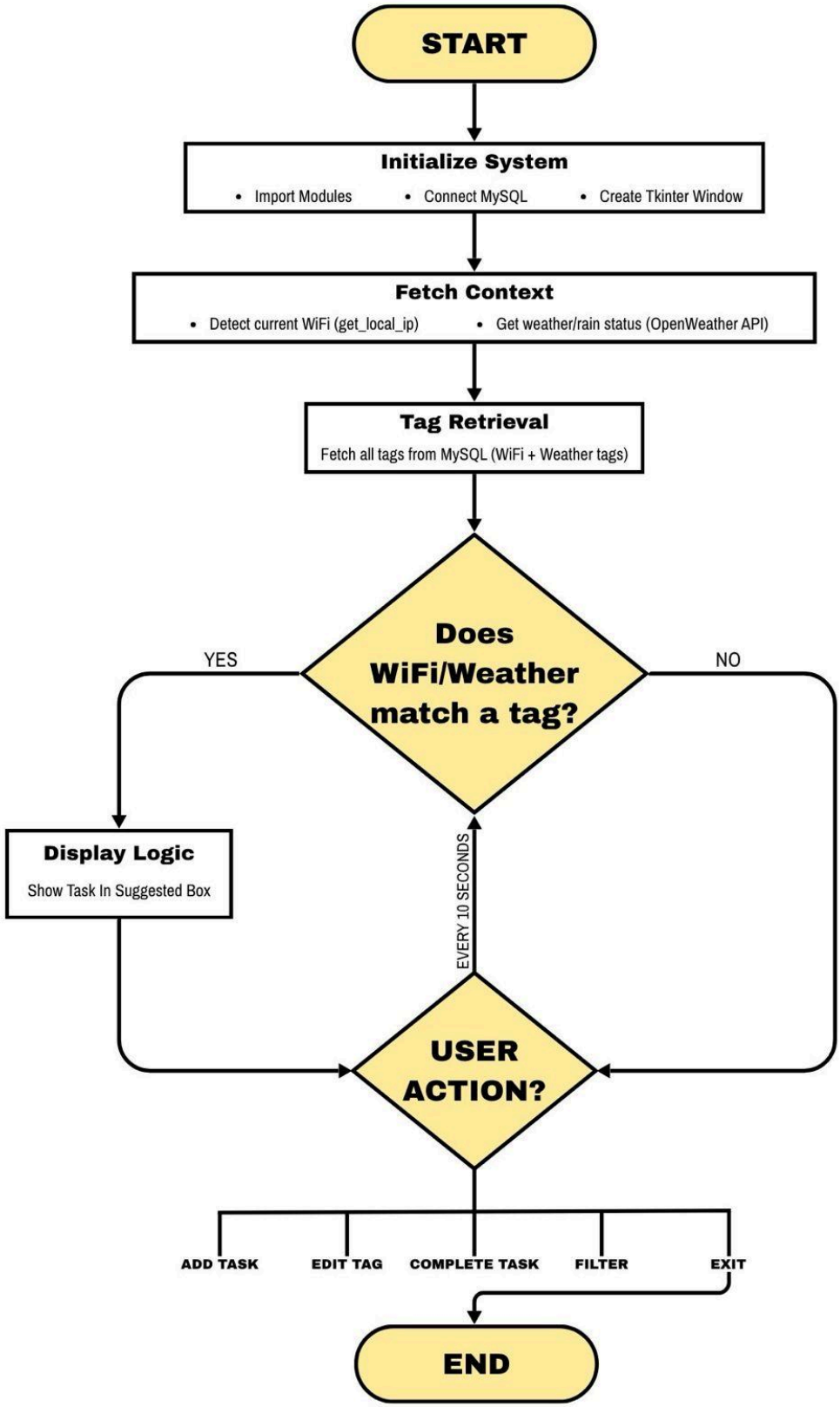
- **Language:** Python
- **Libraries:** tkinter, PIL, mysql.connector, socket, threading, webbrowser, time, requests, Flask, request, jsonify, render_template_string
- **APIs:** GoogleMaps, OpenWeatherMap
- **Environment:** IDLE
- **Operating System:** MacOS

ARCHITECTURE & DESIGN

The Prioris architecture is built around modular, independent components that work together to provide real-time, adaptive suggestions. Each module communicates through a central controller that continuously updates the Suggested Task Box.

3.1 Architecture Overview

- **User Interface Layer:** Provides interaction through Tkinter, displaying task lists, the Eisenhower grid, and the Suggested Task Box.
- **Logic Layer:** Responsible for context detection, data fetching, and decision-making.
- **Database Layer:** Handles storage of task details, priority levels, and user-defined Wi-Fi contexts.
- **Suggestion Engine:** The heart of Prioris — this algorithm dynamically picks and displays the most relevant task using data from all modules.


START
Initialize System
• Import Modules
• Connect MySQL
• Create Tkinter Window
Fetch Context
• Detect current WiFi (get_local_ip)
• Get weather/rain status (OpenWeather API)
Tag Retrieval
Fetch all tags from MySQL (WiFi + Weather tags)
Does WiFi/Weather match a tag?
YES
NO
Display Logic
Show Task In Suggested Box
EVERY 10 SECONDS
USER ACTION?
ADD TASK
EDIT TAG
COMPLETE TASK
FILTER
EXIT
END

3.2 Core Modules

- **Task Manager:** Handles task CRUD operations.
- **Eisenhower Engine:** Evaluates importance and urgency to sort tasks into quadrants.
- **Wi-Fi Context Module:** Detects connected Wi-Fi and sets active context.
- **Weather Module:** Retrieves real-time weather data and influences task relevance.
- **Deadline Engine:** Checks due dates and urgency thresholds.
- **Suggestion Engine:** Combines results from all modules to display the top recommended tasks in the Suggested Task Box.

IMPLEMENTATION

```
import tkinter as tk
from tkinter import ttk, messagebox
from PIL import Image, ImageTk
import mysql.connector
from datetime import date, datetime, timedelta
import socket, threading, webbrowser, time, requests
from flask import Flask, request, jsonify, render_template_string

API_KEY = "AIzaSyAajd58n7_B6YY94QWEBGsjB4RbASIFdE8"

# -------------------- MySQL Connection --------------------
connection = mysql.connector.connect(
    host="localhost",
    user="root",
    password="new_password",
    database="prioris"
)
cursor = connection.cursor(buffered=True)
print("✅ Connected to MySQL!")

# -------------------- Tkinter Window --------------------
root = tk.Tk()
root.title("Prioris - Task Manager")
root.geometry("1300x900")
root.config(bg="#DEDEDE")

canvas = tk.Canvas(root, bg="#F3F4F6", highlightthickness=0)
canvas.pack(fill="both", expand=True)

# -------------------- Logo --------------------
img = Image.open("/Users/tanishk/Downloads/prioris_logo.png")
img = img.resize((250, 110))
photo = ImageTk.PhotoImage(img)
canvas.create_image(25, 5, image=photo, anchor="nw")
canvas.image = photo

# -------------------- Rounded Rectangle Function --------------------
def draw_rounded_rect(canvas, x1, y1, x2, y2, radius=25, color="#1555DC", outline=None, width=2):
    points = [
        x1+radius, y1, x2-radius, y1,
        x2, y1, x2, y1+radius,
        x2, y2-radius, x2, y2,
        x2-radius, y2, x1+radius, y2,
        x1, y2, x1, y2-radius,
        x1, y1+radius, x1, y1
    ]
    return canvas.create_polygon(points, smooth=True, fill=color,
                                 outline=outline if outline else color, width=width)



# -------------------- Helper: WiFi + City Picker --------------------
def get_local_ip():
    """Return current connected local IP, or None if not connected."""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.connect(("8.8.8.8", 80))
        ip = s.getsockname()[0]
```

```
        s.close()
        return ip
    except Exception:
        return None


def get_city_if_rain_predicted(city_name):
    """Check if rain is predicted in next 24 hours (using OpenWeather FREE API v2.5)."""
    OWM_KEY = "55eff538b31fe122ecc4c64ab9d34be6"

    try:
        # latitude and longitude for the city
        geo_url = f"http://api.openweathermap.org/geo/1.0/direct?q={city_name}&limit=1&appid={OWM_KEY}"
        geo_resp = requests.get(geo_url, timeout=8).json()
        if not geo_resp:
            print(f"⚠️ City not found: {city_name}")
            return False

        lat, lon = geo_resp[0]["lat"], geo_resp[0]["lon"]

        # 5-day / 3-hour forecast
        url = f"http://api.openweathermap.org/data/2.5/forecast?lat={lat}&lon={lon}&appid={OWM_KEY}&units=metric"
        resp = requests.get(url, timeout=8).json()
        if "list" not in resp:
            print(f"⚠️ Weather data unavailable for {city_name}")
            return False

        # next 8 forecast entries (≈24 hours)
        for entry in resp["list"][:8]:
            weather_desc = entry["weather"][0]["main"].lower()
            rain_amt = entry.get("rain", {}).get("3h", 0)
            if "rain" in weather_desc or rain_amt > 0:
                print(f"🌧️ Rain predicted in {city_name}")
                return True

        print(f"☀️ No rain predicted in {city_name}")
        return False

    except Exception as e:
        print("Weather check failed:", e)
        return False


def pick_city(API_KEY, port=5051):
    app = Flask(__name__)
    result = {"city": None}

    HTML = """
    <!doctype html>
    <html><head>
      <meta charset="utf-8">
      <title>Pick Location</title>
      <style>html,body,#map{height:100%;margin:0;padding:0}</style>
    </head>
    <body><div id="map"></div>
    <script>
      let map, marker;
      function initMap() {
        map = new google.maps.Map(document.getElementById('map'), {
          center:{lat:28.6,lng:77.2},zoom:6
        });
        map.addListener('click', e=>{
          if(marker)marker.setMap(null);
          marker=new google.maps.Marker({position:e.latLng,map});
```

```
      const lat=e.latLng.lat(),lng=e.latLng.lng();
      fetch(`https://maps.googleapis.com/maps/api/geocode/json?latlng=${lat},${lng}&key={{k}}`)
      .then(r=>r.json()).then(j=>{
        let c=null;
        for(const r1 of (j.results||[]))
          for(const a of (r1.address_components||[]))
            if(a.types.includes("locality")){c=a.long_name;break;}
        fetch('/set',{method:'POST',headers:{'Content-Type':'application/json'},
          body:JSON.stringify({city:c})});
      });
    });
  }
</script>
<script src="https://maps.googleapis.com/maps/api/js?key={{k}}&callback=initMap" async defer></script>
</body></html>
"""

@app.route("/")
def index(): return render_template_string(HTML, k=API_KEY)

@app.route("/set", methods=["POST"])
def set_city():
    data = request.get_json()
    result["city"] = data.get("city")
    try: requests.get(f"http://127.0.0.1:{port}/shutdown", timeout=1)
    except: pass
    return jsonify(ok=True)

@app.route("/shutdown")
def shutdown():
    func = request.environ.get("werkzeug.server.shutdown")
    if func: func()
    return "ok"

def run(): app.run(port=port, debug=False, use_reloader=False)
threading.Thread(target=run, daemon=True).start()
webbrowser.open(f"http://127.0.0.1:{port}/")

while result["city"] is None:
    time.sleep(0.2)
return result["city"]


def update_suggested_tasks():
    """Fetch and display suggested tasks based on WiFi and Weather (rain prediction)."""
    wifi = get_local_ip()
    wifi_tags = []
    sql = "SELECT tagname FROM tags WHERE wifi=%s"
    cursor.execute(sql, (wifi,))
    result = cursor.fetchall()
    for i in result:
        wifi_tags.append(i[0])

    for widget in scrollable_frame.winfo_children():
        widget.destroy()

    found_any = False
    total_tasks = 0

    # --- Helper to create pill UI for each task ---
    def create_task_pill(text):
        pill_frame = tk.Frame(scrollable_frame, bg="#1555DC")
        pill_frame.pack(fill="x", padx=10, pady=8)
```

```
    canvas_pill = tk.Canvas(
        pill_frame,
        width=300,
        height=60,
        bg="#1555DC",
        highlightthickness=0,
        bd=0
    )
    canvas_pill.pack()

    # Rounded translucent pill
    x1, y1, x2, y2, r = 5, 5, 295, 55, 20
    canvas_pill.create_polygon(
        x1+r, y1, x2-r, y1, x2, y1, x2, y1+r,
        x2, y2-r, x2, y2, x2-r, y2, x1+r, y2,
        x1, y2, x1, y2-r, x1, y1+r, x1, y1,
        smooth=True,
        fill="#BFD3FF",
        outline="#BFD3FF"
    )
    canvas_pill.create_text(
        (x1 + x2) / 2,
        (y1 + y2) / 2,
        text=text,
        font=("Poppins", 20, "bold"),
        fill="#003366"
    )

# --- WiFi-based Suggested Tasks ---
for j in wifi_tags:
    sql = "SELECT taskName FROM taskinfo WHERE taskStatus=0 AND tag=%s"
    cursor.execute(sql, (j,))
    result = cursor.fetchall()
    for k in result:
        create_task_pill(k[0])
        total_tasks += 1
        found_any = True

# --- Weather-based Suggested Tasks (rain prediction) ---
rain_tags = []
cursor.execute("SELECT location FROM tags")
result = cursor.fetchall()
for i in result:
    if i[0] is not None and i[0] != "":
        if get_city_if_rain_predicted(i[0]):
            sql = "SELECT tagname FROM tags WHERE location=%s"
            cursor.execute(sql, (i[0],))
            result2 = cursor.fetchall()
            for m in result2:
                rain_tags.append(m[0])

rain_tags = list(dict.fromkeys(rain_tags))

for j in rain_tags:
    sql = "SELECT taskName FROM taskinfo WHERE taskStatus=0 AND tag=%s"
    cursor.execute(sql, (j,))
    result = cursor.fetchall()
    for k in result:
        create_task_pill(k[0])
        total_tasks += 1
        found_any = True

# --- No task message ---
if not found_any:
```

```
        tk.Label(
            scrollable_frame,
            text="No suggested tasks",
            font=("Poppins", 16, "italic"),
            fg="white",
            bg="#1555DC"
        ).pack(pady=20)

    # --- Enable scrollbar only if >4 tasks ---
    if total_tasks > 4:
        scrollbar.pack(side="right", fill="y")
        suggested_canvas.configure(yscrollcommand=scrollbar.set)
    else:
        scrollbar.pack_forget()
        suggested_canvas.configure(yscrollcommand=None)
def monitor_wifi():
    """Refresh suggested tasks safely every 10 seconds using Tkinter's main thread."""
    wifi = get_local_ip()
    update_suggested_tasks()

    root.after(10000, monitor_wifi)



# ---------- Blue Sidebar ----------
draw_rounded_rect(canvas, 25, 125, 400, 550, radius=40, color="#1555DC")
canvas.create_text(160, 160, text="Suggested Tasks",
                   font=("Poppins", 28, "bold"), fill="white")


# ---------- Suggested Task Box (Scrollable Frame) ----------
suggested_box_frame = tk.Frame(root, bg="#1555DC")
suggested_box_frame.place(x=45, y=200, width=340, height=320)

#canvas inside the blue sidebar
suggested_canvas = tk.Canvas(
    suggested_box_frame,
    bg="#1555DC",
    highlightthickness=0,
    bd=0
)
suggested_canvas.pack(side="left", fill="both", expand=True)

# Custom blue scrollbar
style = ttk.Style()
style.theme_use("clam")
style.configure(
    "Blue.Vertical.TScrollbar",
    gripcount=0,
    background="#1555DC",
    darkcolor="#1555DC",
    lightcolor="#1555DC",
    troughcolor="#1555DC",
    bordercolor="#1555DC",
    arrowcolor="#1555DC"
)

scrollbar = ttk.Scrollbar(
    suggested_box_frame,
    orient="vertical",
    command=suggested_canvas.yview,
    style="Blue.Vertical.TScrollbar"
)
scrollbar.pack(side="right", fill="y")
```

```
suggested_canvas.configure(yscrollcommand=scrollbar.set)

# Inner frame to hold translucent pills
scrollable_frame = tk.Frame(suggested_canvas, bg="#1555DC")
suggested_canvas.create_window((0, 0), window=scrollable_frame, anchor="nw")

# Update scroll region automatically
def on_frame_configure(event):
    suggested_canvas.configure(scrollregion=suggested_canvas.bbox("all"))
scrollable_frame.bind("<Configure>", on_frame_configure)

# Mousewheel scrolling support
def _on_mousewheel(event):
    if root.tk.call('tk', 'windowingsystem') == 'aqua':  # macOS
        suggested_canvas.yview_scroll(-1 * int(event.delta), "units")
    else:
        suggested_canvas.yview_scroll(-1 * int(event.delta / 120), "units")

scrollable_frame.bind_all("<MouseWheel>", _on_mousewheel)

# ---------- Filter Buttons ----------
filter_buttons = [
    {"text": "All", "x1": 425, "y1": 40, "x2": 520, "y2": 80},
    {"text": "Today", "x1": 530, "y1": 40, "x2": 630, "y2": 80},
    {"text": "Completed", "x1": 1140, "y1": 40, "x2": 1280, "y2": 80}
]
active_button = 0

for i, btn in enumerate(filter_buttons):
    color = "#1555DC" if i == active_button else "#FFFFFF"
    text_color = "white" if i == active_button else "black"
    btn["shape_id"] = draw_rounded_rect(canvas, btn["x1"], btn["y1"], btn["x2"], btn["y2"],
                                        radius=35, color=color, outline="#DEDEDE")
    btn["text_id"] = canvas.create_text(
        (btn["x1"] + btn["x2"]) / 2,
        (btn["y1"] + btn["y2"]) / 2,
        text=btn["text"],
        font=("Poppins", 20, "bold"),
        fill=text_color
    )

# ---------- Eisenhower Boxes ----------
box_width, box_height = 420, 350
padding_x, padding_y, gap, corner_radius = 425, 100, 15, 30
boxes = [
    (padding_x, padding_y, "❶ Urgent & Important", "#FC0001"),
    (padding_x + box_width + gap, padding_y, "❷ Not Urgent but Important", "#F37200"),
    (padding_x, padding_y + box_height + gap, "❸ Urgent but Not Important", "#1964F4"),
    (padding_x + box_width + gap, padding_y + box_height + gap, "❹ Not Urgent & Not Important", "#2FBD54")
]
for (x, y, text, color) in boxes:
    draw_rounded_rect(canvas, x, y, x + box_width, y + box_height,
                      radius=corner_radius, color="white", outline="#DEDEDE", width=2)
    canvas.create_text(x + 15, y + 30, text=text,
                       font=("Poppins", 25, "bold"),
                       fill=color, anchor="w")

positions = {
    1: (padding_x + 20, padding_y + 80),
    2: (padding_x + box_width + gap + 20, padding_y + 80),
    3: (padding_x + 20, padding_y + box_height + gap + 80),
    4: (padding_x + box_width + gap + 20, padding_y + box_height + gap + 80)
```

```
}
task_text_ids = {1: [], 2: [], 3: [], 4: []}
deadline_text_ids = {1: [], 2: [], 3: [], 4: []}
checkbox_ids = {}
tick_ids = {}
delete_icon_ids = {1: [], 2: [], 3: [], 4: []}


DEADLINE_RIGHT_PAD = 16
deadline_x = {
    1: padding_x + box_width - DEADLINE_RIGHT_PAD,
    2: (padding_x + box_width + gap) + box_width - DEADLINE_RIGHT_PAD,
    3: padding_x + box_width - DEADLINE_RIGHT_PAD,
    4: (padding_x + box_width + gap) + box_width - DEADLINE_RIGHT_PAD,
}

def clear_matrix():
    for cat in task_text_ids:
        for tid in task_text_ids[cat]:
            canvas.delete(tid)
        task_text_ids[cat].clear()
    for cat in deadline_text_ids:
        for did in deadline_text_ids[cat]:
            canvas.delete(did)
        deadline_text_ids[cat].clear()
    for cat in checkbox_ids:
        for cid in checkbox_ids[cat]:
            canvas.delete(cid)
    checkbox_ids.clear()
    for cat in tick_ids:
        for kid in tick_ids[cat]:
            canvas.delete(kid)
    tick_ids.clear()
    for cat in delete_icon_ids:
        for did in delete_icon_ids[cat]:
            canvas.delete(did)
    delete_icon_ids.clear()

# -------------------- Animations --------------------
def animate_checkbox(rect_id, start_color, end_color, steps=10, delay=30):
    def hex_lerp(a, b, t): return int(a + (b - a) * t)
    def lerp_color(c1, c2, t):
        c1 = canvas.winfo_rgb(c1)
        c2 = canvas.winfo_rgb(c2)
        return "#%02x%02x%02x" % (
            hex_lerp(c1[0] // 256, c2[0] // 256, t),
            hex_lerp(c1[1] // 256, c2[1] // 256, t),
            hex_lerp(c1[2] // 256, c2[2] // 256, t)
        )
    for i in range(steps + 1):
        color = lerp_color(start_color, end_color, i / steps)
        root.after(i * delay, lambda c=color: canvas.itemconfig(rect_id, outline=c))

def animate_tick(tick_id, show=True, steps=10, delay=30):
    bg_rgb = (243, 244, 246)
    green_rgb = (0, 170, 0)
    start = bg_rgb if show else green_rgb
    end = green_rgb if show else bg_rgb
    def lerp(a, b, t): return int(a + (b - a) * t)
    for i in range(steps + 1):
        t = i / steps
        r, g, b = lerp(start[0], end[0], t), lerp(start[1], end[1], t), lerp(start[2], end[2], t)
        color = f"#{r:02x}{g:02x}{b:02x}"
        root.after(i * delay, lambda c=color: canvas.itemconfig(tick_id, fill=c))
```

```
# -------------------- DB Helpers --------------------
def toggle_status(task_name, current_status):
    new_status = 0 if current_status == 1 else 1
    with connection.cursor(buffered=True) as cur:
        cur.execute("UPDATE taskinfo SET taskStatus=%s WHERE taskName=%s", (new_status, task_name))
    connection.commit()
    print(f"✅ {task_name} → {new_status}")
    current_filter = filter_buttons[active_button]["text"]
    displayTask(current_filter, smooth=True)

def delete_task(task_name):
    with connection.cursor(buffered=True) as cur:
        cur.execute("DELETE FROM taskinfo WHERE taskName=%s", (task_name,))
    connection.commit()
    print(f"🗑️ Deleted task → {task_name}")
    current_filter = filter_buttons[active_button]["text"]
    displayTask(current_filter, smooth=True)

ANIM_STEPS = 30
ANIM_DELAY = 20

MONTH_FIX = {"Sep": "Sept"}
def format_deadline(d):
    if not d:
        return ""
    if isinstance(d, datetime):
        d = d.date()
    try:
        mon = d.strftime("%b")
    except Exception:
        mon = ["Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct","Nov","Dec"][d.month-1]
    mon = MONTH_FIX.get(mon, mon)
    return f"{d.day} {mon}"




# -------------------- Tag Manager Window --------------------
def open_tag_manager_popup():
    popup = tk.Toplevel(root)
    popup.title("Manage Tags")
    popup.geometry("500x350")
    popup.configure(bg="#FFFFFF")
    popup.grab_set()

    # ---- Title ----
    tk.Label(
        popup,
        text="Your Tags",
        font=("Poppins", 28, "bold"),
        bg="#FFFFFF",
        fg="#1555DC"
    ).pack(pady=(22, 12))

    # ---- Header ----
    header = tk.Frame(popup, bg="#F3F4F6", highlightbackground="#E0E0E0", highlightthickness=1)
    header.pack(fill="x", padx=35, pady=(0, 8))
    header.grid_columnconfigure(0, weight=3, uniform="col")
    header.grid_columnconfigure(1, weight=2, uniform="col")
    header.grid_columnconfigure(2, weight=1, uniform="col")
```

```
tk.Label(header, text="Tag Name", font=("Poppins", 16, "bold"),
        bg="#F3F4F6", fg="#000000", anchor="w", padx=12).grid(row=0, column=0, sticky="we")
tk.Label(header, text="Type", font=("Poppins", 16, "bold"),
        bg="#F3F4F6", fg="#000000", anchor="center").grid(row=0, column=1, sticky="we")
tk.Label(header, text="Delete", font=("Poppins", 16, "bold"),
        bg="#F3F4F6", fg="#000000", anchor="center").grid(row=0, column=2, sticky="we")

# ---- Table container ----
table_frame = tk.Frame(popup, bg="#FFFFFF")
table_frame.pack(fill="both", expand=True, padx=35, pady=(0, 70))
table_frame.grid_columnconfigure(0, weight=3, uniform="col")
table_frame.grid_columnconfigure(1, weight=2, uniform="col")
table_frame.grid_columnconfigure(2, weight=1, uniform="col")

# ---- Load tags ----
def load_tags():
    for w in table_frame.winfo_children():
        w.destroy()

    cursor.execute("SELECT tagID, tagName, wifi, location FROM TAGS")
    tags = cursor.fetchall()

    if not tags:
        tk.Label(table_frame, text="No tags defined yet.",
                 font=("Poppins", 14), bg="#FFFFFF", fg="#888888").pack(pady=20)
        return

    for i, (tid, name, wifi, loc) in enumerate(tags):
        tag_type = "WiFi" if wifi else "Weather"
        bg_color = "#FAFAFA" if i % 2 == 0 else "#FFFFFF"

        name_lbl = tk.Label(table_frame, text=name, font=("Poppins", 15,"bold"),
                            bg=bg_color, fg="#000000", anchor="w", padx=12)
        type_lbl = tk.Label(table_frame, text=tag_type, font=("Poppins", 14),
                            bg=bg_color, fg="#000000", anchor="center")
        del_btn = tk.Label(table_frame, text="✖", fg="#E11D48",
                           font=("Poppins", 18, "bold"), bg=bg_color,
                           anchor="center", cursor="hand2")

        name_lbl.grid(row=i, column=0, sticky="we", pady=2, ipady=6)
        type_lbl.grid(row=i, column=1, sticky="we", pady=2, ipady=6)
        del_btn.grid(row=i, column=2, sticky="we", pady=2, ipady=6)

        def delete_tag(tid=tid, name=name):
            if messagebox.askyesno("Confirm Delete", f"Delete tag '{name}'?"):
                cursor.execute("DELETE FROM TAGS WHERE tagID=%s", (tid,))
                connection.commit()
                load_tags()

        del_btn.bind("<Button-1>", lambda e, tid=tid, name=name: delete_tag(tid, name))

load_tags()

# ---- Floating Add Tag button ----
add_btn_canvas = tk.Canvas(popup, width=100, height=35, bg="#FFFFFF", highlightthickness=0)
add_btn_canvas.place(relx=0.88, rely=0.9, anchor="center")

pill_id = draw_rounded_rect(add_btn_canvas, 0, 0, 100, 35,
                            radius=35, color="#1555DC", outline="#1555DC")
text_id = add_btn_canvas.create_text(50, 17, text="+ Add Tag",
                                     fill="white", font=("Poppins", 18, "bold"))

def open_add_tag(_=None):
    popup.destroy()
```

```
        open_edit_tag_popup()

    for tag in (pill_id, text_id):
        add_btn_canvas.tag_bind(tag, "<Button-1>", open_add_tag)




def open_edit_tag_popup():
    popup = tk.Toplevel(root)
    popup.title("Add / Edit Tag")
    popup.geometry("500x400")
    popup.configure(bg="#DEDEDE")
    popup.grab_set()

    tk.Label(popup, text="Enter Tag Name:", font=("Poppins", 22, "bold"),
             bg="#DEDEDE",fg="#1555DC").pack(pady=(30, 8))
    tag_entry = tk.Entry(popup, font=("Poppins", 18), width=25,bg="#5A565F")
    tag_entry.pack(pady=(0, 20), ipady=6)

    tk.Label(popup, text="Tag Type:", font=("Poppins", 22, "bold"),
             bg="#DEDEDE",fg="#1555DC").pack(pady=(10, 8))
    tag_type = ttk.Combobox(popup, values=["WiFi", "Weather"],
                            state="readonly", font=("Poppins", 16))
    tag_type.pack(pady=(0, 25))
    tag_type.set("Select Type")

    def save_tag():
        tag = tag_entry.get().strip()
        tp = tag_type.get()

        if not tag or tp == "Select Type":
            messagebox.showerror("Error", "Please enter name and type.")
            return

        if tp == "WiFi":
            wifi = get_local_ip()
            if not wifi:
                messagebox.showerror("Error", "WiFi not detected!")
                return
            val = (tag, wifi)
            sql = "INSERT INTO TAGS (tagname, wifi) VALUES (%s, %s)"
            cursor.execute(sql, val)
            connection.commit()
            messagebox.showinfo("Success", f"Tag '{tag}' added (WiFi linked).")
            popup.destroy()

        elif tp == "Weather":
            messagebox.showinfo("Info", "Select your city on the map.")
            city = pick_city(API_KEY, port=5051)
            val = (tag, city)
            sql = "INSERT INTO TAGS (tagname, location) VALUES (%s, %s)"
            cursor.execute(sql, val)
            connection.commit()
            messagebox.showinfo("Success", f"Tag '{tag}' added for city: {city}")
            popup.destroy()

    # ---- Rounded blue Save button  ----
    btn_holder = tk.Frame(popup, bg="#DEDEDE")
    btn_holder.pack(pady=16)
    pill_w, pill_h = 150, 50
    btn_canvas = tk.Canvas(btn_holder, width=pill_w, height=pill_h,
                           bg="#DEDEDE", highlightthickness=0)
    btn_canvas.pack()
```

```
    pill_id = draw_rounded_rect(btn_canvas, 0, 0, pill_w, pill_h,
                                radius=40, color="#1555DC", outline="#1555DC")
    text_id = btn_canvas.create_text(pill_w/2, pill_h/2,
                                     text="Save", fill="white",
                                     font=("Poppins", 20, "bold"))
    for tag in (pill_id, text_id):
        btn_canvas.tag_bind(tag, "<Button-1>", lambda e: save_tag())
    popup.bind("<Return>", lambda e: save_tag())




# -------------------- Add Task Popup --------------------

def open_add_task_popup():
    popup = tk.Toplevel(root)
    popup.title("Add Task")
    popup.geometry("600x400")
    popup.configure(bg="#FFFFFF")
    popup.grab_set()

    # --- Task Name ---
    name_entry = tk.Entry(
        popup,
        font=("Poppins", 50, "bold"),
        width=40,
        relief="flat",
        bd=0,
        highlightthickness=0,
        highlightbackground="#FFFFFF",
        highlightcolor="#FFFFFF",
        bg="#FFFFFF",
        fg="gray",
        insertbackground="black"
    )
    name_entry.insert(0, "Enter Task")
    name_entry.pack(padx=24, pady=(30, 12), ipady=6, ipadx=6)

    def on_entry_click(event):
        if name_entry.get() == "Enter Task":
            name_entry.delete(0, "end")
            name_entry.config(fg="black")

    def on_focusout(event):
        if name_entry.get().strip() == "":
            name_entry.insert(0, "Enter Task")
            name_entry.config(fg="gray")

    name_entry.bind("<FocusIn>", on_entry_click)
    name_entry.bind("<FocusOut>", on_focusout)


    form = tk.Frame(popup, bg="#FFFFFF")
    form.pack(fill="x", padx=24, pady=(4, 6))
    form.grid_columnconfigure(0, weight=0)
    form.grid_columnconfigure(1, weight=1, uniform="in")
    form.grid_columnconfigure(2, weight=1, uniform="in")

    style = ttk.Style(popup)
    style.theme_use("clam")
    style.configure(
        "Popup.TCombobox",
        padding=6,
        arrowsize=12,
```

```
    fieldbackground="#FFFFFF",
    background="#FFFFFF",
    foreground="#000000",
    arrowcolor="#000000"
)
popup.option_add("*TCombobox*Listbox.background", "#FFFFFF")
popup.option_add("*TCombobox*Listbox.foreground", "#000000")
popup.option_add("*TCombobox*Listbox.selectBackground", "#E5E7EB")
popup.option_add("*TCombobox*Listbox.selectForeground", "#000000")

CB_FONT = ("Poppins", 12)

# ---- Deadline ----
tk.Label(form, text="Deadline:", font=("Poppins", 25, "bold"),
         bg="#FFFFFF", fg="#000000").grid(row=0, column=0, sticky="w", pady=(0, 8))

today_date = datetime.today().date()
date_options = ["Date"] + [(today_date + timedelta(days=i)).strftime("%Y-%m-%d") for i in range(365)]
time_options = ["Time"] + [f"{h:02d}:{m:02d}" for h in range(24) for m in (0, 30)]

date_var = tk.StringVar()
time_var = tk.StringVar()

date_box = ttk.Combobox(form, textvariable=date_var, state="readonly",
                        font=CB_FONT, style="Popup.TCombobox", values=date_options)
date_box.set("Date")
date_box.grid(row=0, column=1, sticky="we", padx=(8, 4), pady=(0, 12))

time_box = ttk.Combobox(form, textvariable=time_var, state="readonly",
                        font=CB_FONT, style="Popup.TCombobox", values=time_options)
time_box.set("Time")
time_box.grid(row=0, column=2, sticky="we", padx=(4, 2), pady=(0, 12))

# ---- Category ----
tk.Label(form, text="Category:", font=("Poppins", 25, "bold"),
         bg="#FFFFFF", fg="#000000").grid(row=1, column=0, sticky="w", pady=(8, 10))

cat_var = tk.StringVar()
category_values = [
    "1 - Urgent & Important",
    "2 - Not Urgent but Important",
    "3 - Urgent but Not Important",
    "4 - Not Urgent & Not Important",
]
cat_box = ttk.Combobox(form, textvariable=cat_var, state="readonly",
                       font=CB_FONT, style="Popup.TCombobox", values=category_values)
cat_box.set("Select Category")
cat_box.grid(row=1, column=1, columnspan=2, sticky="we", padx=(8, 4), pady=(8, 12))

# ---- Tag ----
tk.Label(form, text="Tag:", font=("Poppins", 25, "bold"),
         bg="#FFFFFF", fg="#000000").grid(row=2, column=0, sticky="w", pady=(8, 10))

cursor.execute("SELECT tagName, tagID FROM Tags")
tags = cursor.fetchall()
tag_map = {f"{tname} (#{tid})": tname for tname, tid in tags}
tag_values = list(tag_map.keys())
tag_var = tk.StringVar()

tag_box = ttk.Combobox(form, textvariable=tag_var, state="readonly",
                       font=CB_FONT, style="Popup.TCombobox", values=tag_values)
tag_box.set("Select Tag")
tag_box.grid(row=2, column=1, columnspan=2, sticky="we", padx=(8, 4), pady=(8, 12))
```

```
# ---- Save ----
def save_task():
    task_name = name_entry.get().strip()
    if not task_name or task_name == "Enter Task":
        messagebox.showerror("Error", "Please enter a task name.")
        return

    if cat_var.get() == "Select Category":
        messagebox.showerror("Error", "Please choose a category.")
        return

    try:
        category = int(cat_var.get().split(" - ")[0])
    except Exception:
        category = 1

    if tag_var.get() == "Select Tag":
        tag_name = ""
    else:
        tag_name = tag_map.get(tag_var.get(), "")

    sel_date, sel_time = date_var.get(), time_var.get()
    if sel_date == "Date" or sel_time == "Time":
        messagebox.showerror("Error", "Please select both date and time.")
        return

    try:
        dl = datetime.strptime(f"{sel_date} {sel_time}:00", "%Y-%m-%d %H:%M:%S")
    except ValueError:
        messagebox.showerror("Error", "Invalid date/time selected.")
        return

    sql = "INSERT INTO TaskInfo (taskName, Category, Tag, taskStatus, deadline) VALUES (%s,%s,%s,%s,%s)"
    cursor.execute(sql, (task_name, category, tag_name, 0, dl))
    connection.commit()
    messagebox.showinfo("Success", f"Task '{task_name}' added.")
    popup.destroy()
    displayTask(filter_buttons[active_button]["text"])

    # ----- Add Task pill button -----

btn_holder = tk.Frame(popup, bg="#FFFFFF")
btn_holder.pack(pady=16)

pill_w, pill_h = 100, 30
btn_canvas = tk.Canvas(btn_holder, width=pill_w, height=pill_h,
                       bg="#FFFFFF", highlightthickness=0)
btn_canvas.pack()

# rounded blue pill
pill_id = draw_rounded_rect(btn_canvas, 0, 0, pill_w, pill_h,
                            radius=30, color="#1555DC", outline="#1555DC")

# plus icon and text
text_id = btn_canvas.create_text(50, pill_h//2, text="Save",
                                 fill="white", font=("Poppins", 20, "bold"))

def on_click(_event=None):
    save_task()


for tag in (pill_id,  text_id):
    btn_canvas.tag_bind(tag, "<Button-1>", on_click)
```

```
    popup.bind("<Return>", on_click)

# -------------------- Display Tasks --------------------
def displayTask(filter_type="All", smooth=False):
    clear_matrix()
    cursor.execute("SELECT tagName FROM tags")
    tag_list = [r[0] for r in cursor.fetchall()]
    today = date.today()

    def draw(cat, sql, params=(), show_deadline=False):
        cursor.execute(sql, params)
        result = cursor.fetchall()
        x, y = positions[cat]
        checkbox_ids[cat] = []
        tick_ids[cat] = []
        deadline_text_ids[cat] = []
        delete_icon_ids[cat] = []
        for i, row in enumerate(result):
            if len(row) == 3:
                task_name, status, dl = row
            else:
                task_name, status = row
                dl = None

            box_x, box_y = x, y + i * 28
            size = 14

            rect_id = canvas.create_rectangle(
                box_x, box_y - 6, box_x + size, box_y + size - 6,
                fill="#FFFFFF", outline="#1555DC", width=2
            )
            tick_fill = "#00AA00" if status == 1 else "#F3F4F6"
            tick_id = canvas.create_text(
                box_x + size / 2, box_y - 6 + size / 2,
                text="✓", font=("Poppins", 16, "bold"), fill=tick_fill
            )
            tid = canvas.create_text(
                x + 25, y + i * 28,
                text=task_name,
                font=("Poppins", 20, "bold"),
                fill="gray" if status == 1 else "black",
                anchor="w"
            )

            if show_deadline and dl:
                dtext = format_deadline(dl)
                did = canvas.create_text(
                    deadline_x[cat], y + i * 28,
                    text=dtext, font=("Poppins", 16),
                    fill="#808080", anchor="e"
                )
                deadline_text_ids[cat].append(did)

            def make_cb(name=task_name, st=status, rid=rect_id, tkid=tick_id):
                def handler(event=None):
                    animate_checkbox(rid, "#1555DC", "#1555DC", steps=ANIM_STEPS, delay=ANIM_DELAY)
                    animate_tick(tkid, show=(st == 0), steps=ANIM_STEPS // 2, delay=ANIM_DELAY)
                    root.after(ANIM_STEPS * ANIM_DELAY, lambda: toggle_status(name, st))
                return handler

            cb = make_cb()
            canvas.tag_bind(rect_id, "<Button-1>", cb)
            canvas.tag_bind(tid, "<Button-1>", cb)
```

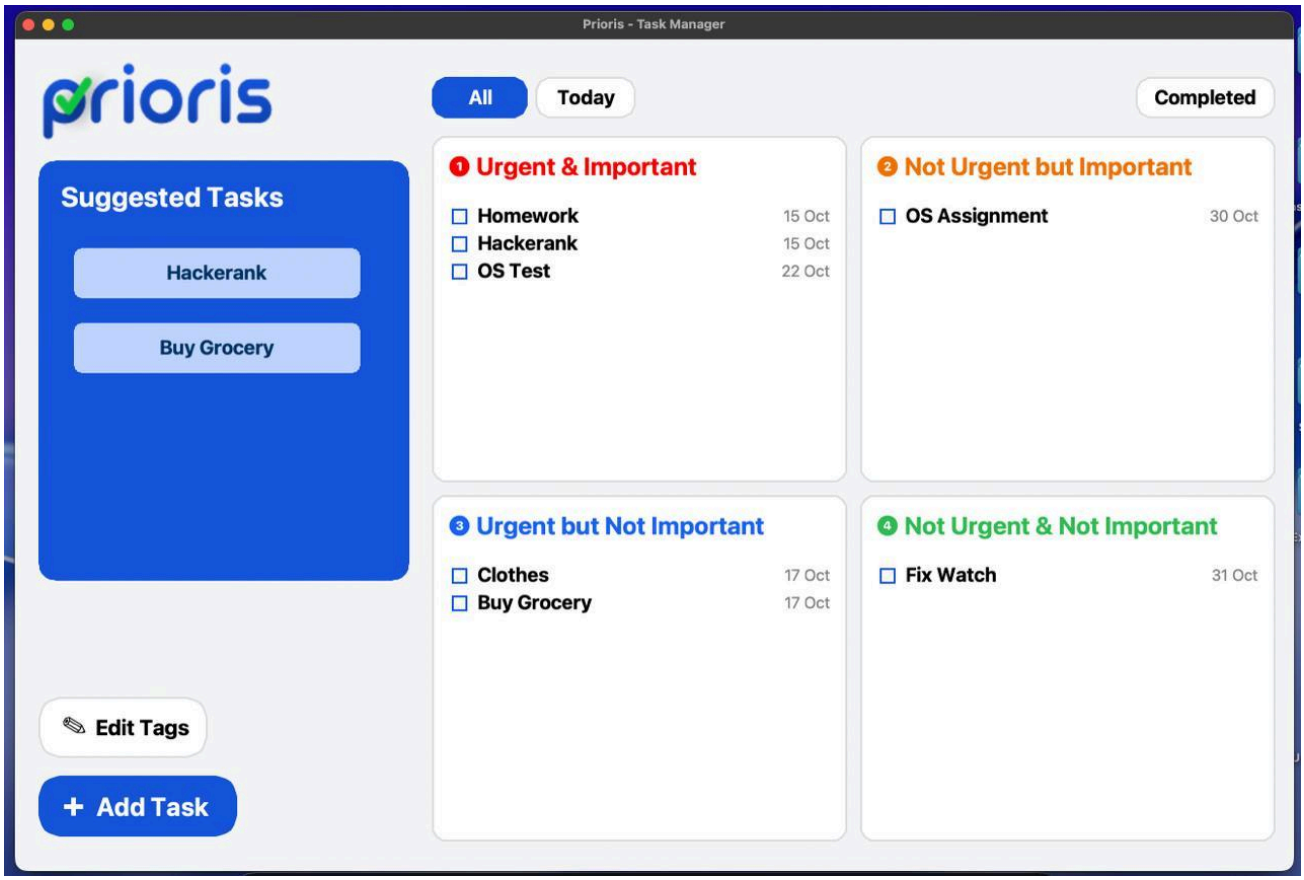
```
            canvas.tag_bind(tick_id, "<Button-1>", cb)


            if filter_type.lower() == "completed":
                if cat in (1, 3):
                    col_left = padding_x
                else:
                    col_left = padding_x + box_width + gap
                x_right = col_left + box_width - 25
                del_id = canvas.create_text(
                    x_right, y + i * 28,
                    text="✖", font=("Poppins", 20, "bold"),
                    fill="#E11D48", anchor="e"
                )
                canvas.tag_bind(del_id, "<Button-1>", lambda e, name=task_name: delete_task(name))
                delete_icon_ids[cat].append(del_id)

            task_text_ids[cat].append(tid)
            checkbox_ids[cat].append(rect_id)
            tick_ids[cat].append(tick_id)

    if filter_type.lower() == "all":
        for c in range(1, 5):
            draw(c, f"SELECT taskName, taskStatus, deadline FROM taskinfo WHERE category={c} AND taskStatus=0
ORDER BY deadline ASC", (), True)
    elif filter_type.lower() == "today":
        for c in range(1, 5):
            draw(c, f"SELECT taskName, taskStatus, deadline FROM taskinfo WHERE category={c} AND taskStatus=0 AND
DATE(deadline)=%s ORDER BY deadline ASC", (date.today(),), True)
    elif filter_type.lower() == "completed":
        for c in range(1, 5):
            draw(c, f"SELECT taskName, taskStatus, deadline FROM taskinfo WHERE category={c} AND taskStatus=1
ORDER BY deadline ASC", (), False)
    else:
        cursor.execute("SELECT tagName FROM tags")
        tag_list = [r[0] for r in cursor.fetchall()]
        if filter_type in tag_list:
            for c in range(1, 5):
                draw(c, f"SELECT taskName, taskStatus FROM taskinfo WHERE category={c} AND taskStatus=0 AND
tag=%s", (filter_type,), False)

# -------------------- Filter Button Logic --------------------
def set_active(index):
    global active_button
    for i, btn in enumerate(filter_buttons):
        if i == index:
            canvas.itemconfig(btn["shape_id"], fill="#1555DC", outline="#1555DC")
            canvas.itemconfig(btn["text_id"], fill="white")
        else:
            canvas.itemconfig(btn["shape_id"], fill="#FFFFFF", outline="#DEDEDE")
            canvas.itemconfig(btn["text_id"], fill="black")
    active_button = index
    filter_type = filter_buttons[index]["text"]
    displayTask(filter_type)
    print(f"Filter → {filter_type}")

for i, btn in enumerate(filter_buttons):
    for tag in (btn["shape_id"], btn["text_id"]):
        canvas.tag_bind(tag, "<Button-1>", lambda e, idx=i: set_active(idx))

# -------------------- Edit + Add Buttons --------------------
draw_rounded_rect(canvas, 25, 670, 195, 730, radius=50, color="#FFFFFF", outline="#DEDEDE")
canvas.create_text(60, 697, text="✎", fill="black", font=("Poppins", 27, "bold"))
canvas.create_text(130, 700, text="Edit Tags", fill="black", font=("Poppins", 22, "bold"))
```

```
draw_rounded_rect(canvas, 25, 750, 225, 810, radius=50, color="#1555DC")
canvas.create_text(60, 777, text="+", fill="white", font=("Poppins", 36, "bold"))
canvas.create_text(140, 780, text="Add Task", fill="white", font=("Poppins", 26, "bold"))


add_task_hotspot = canvas.create_rectangle(25, 750, 225, 810, outline="", fill="")
canvas.tag_bind(add_task_hotspot, "<Button-1>", lambda e: open_add_task_popup())

edit_tag_hotspot = canvas.create_rectangle(25, 670, 195, 730, outline="", fill="")
canvas.tag_bind(edit_tag_hotspot, "<Button-1>", lambda e: open_tag_manager_popup())


# ---------- Initial Load ----------
displayTask("All")
monitor_wifi()
root.mainloop()
```

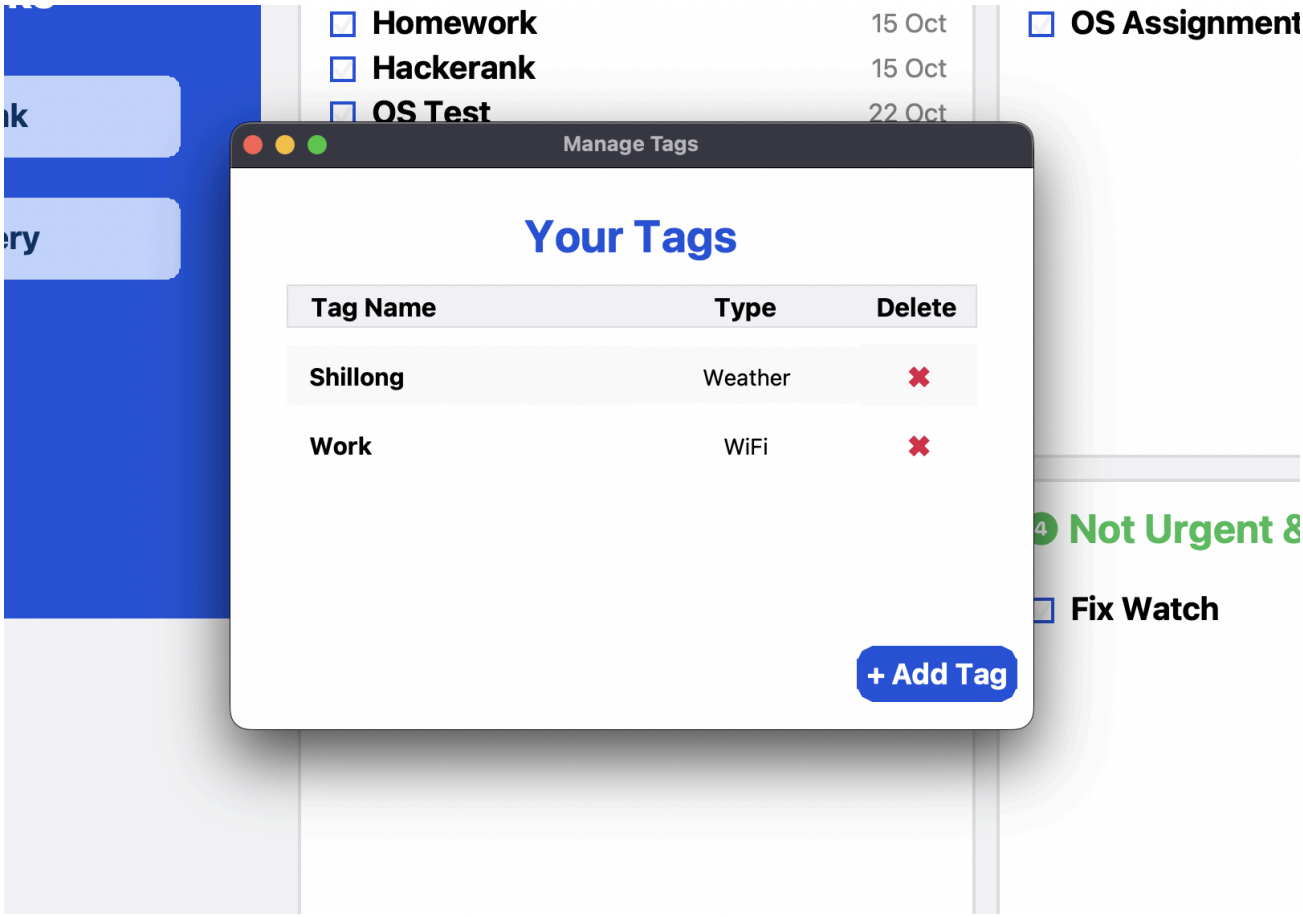
RESULTS AND DISCUSSION

OUTPUT :

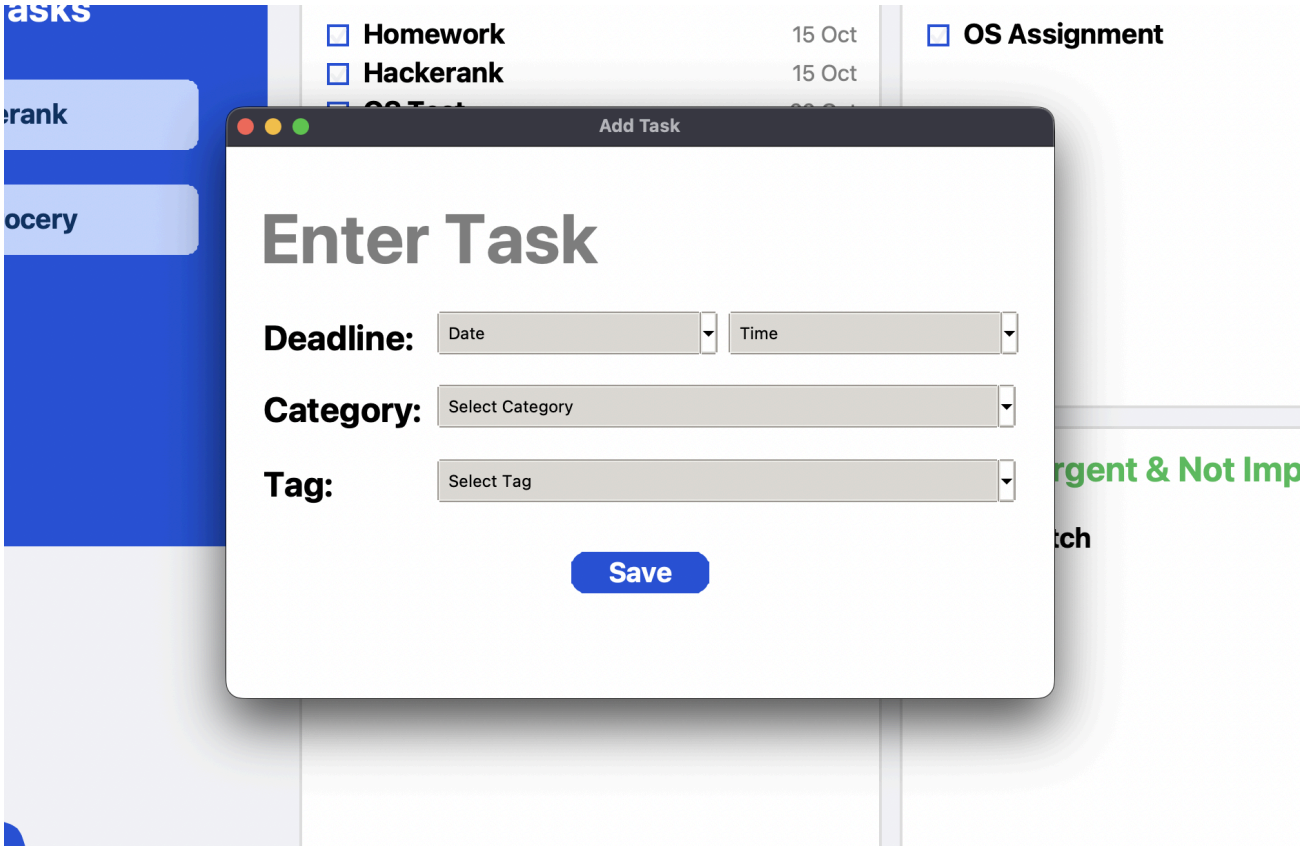
1. **Main window**



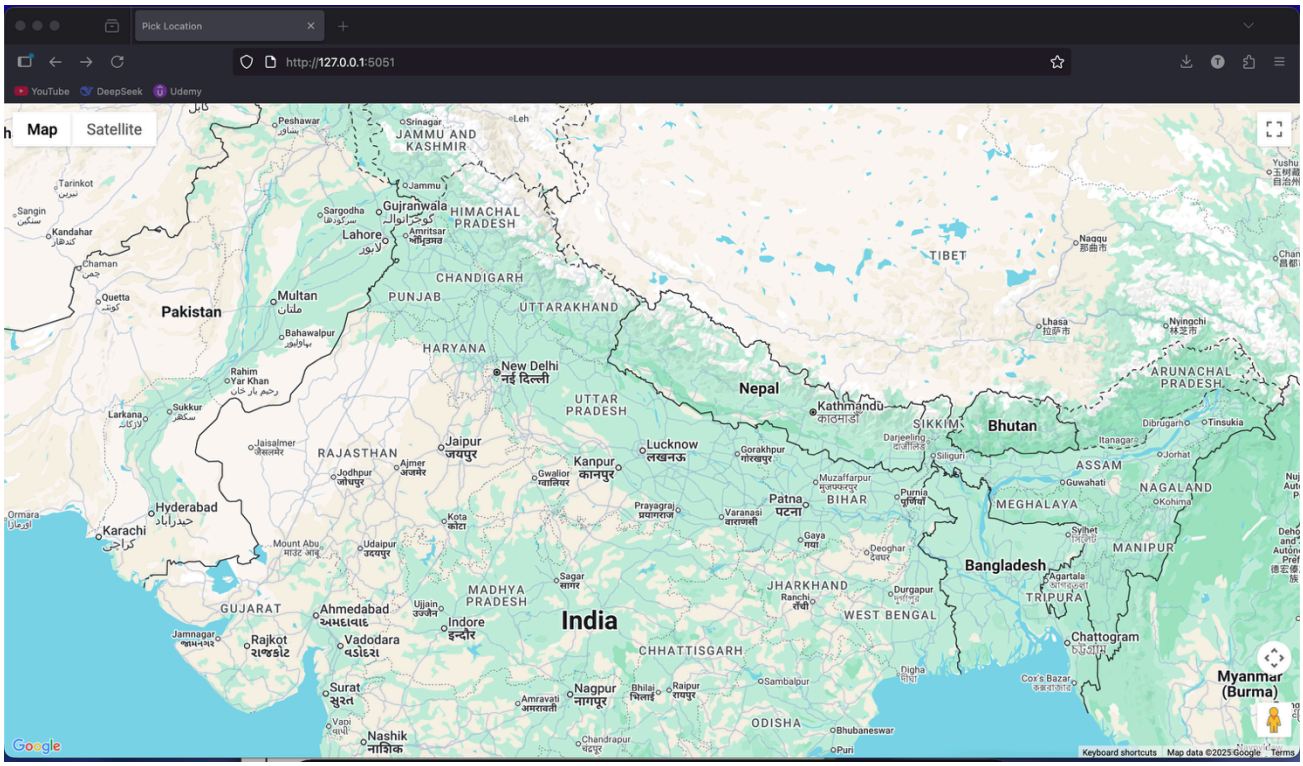
2. **Edit tags window**



3. **Enter Task window**



4. Maps location selection



CONCLUSION

The **PRIORIS** project successfully bridges the gap between conventional task lists and intelligent productivity tools. By combining **context detection**, **Eisenhower prioritization**, and a **Suggested Task Box** powered by real-time conditions like Wi-Fi and weather based on location, Prioris delivers a more human-like and adaptive planning experience.

It encourages users not just to remember tasks, but to act on the *right* ones at the *right* time. The system's modular structure ensures future scalability, allowing integration of features like AI-driven prioritization, motion sensors, and cross-device synchronization.

REFERENCES

1. Tkinter Library – https://docs.python.org/3/library/tkinter.html
2. OpenWeatherMap API – https://openweathermap.org/api
3. Eisenhower Matrix – https://www.eisenhower.me/eisenhower-matrix/
4. MySQL– https://www.mysql.com/